

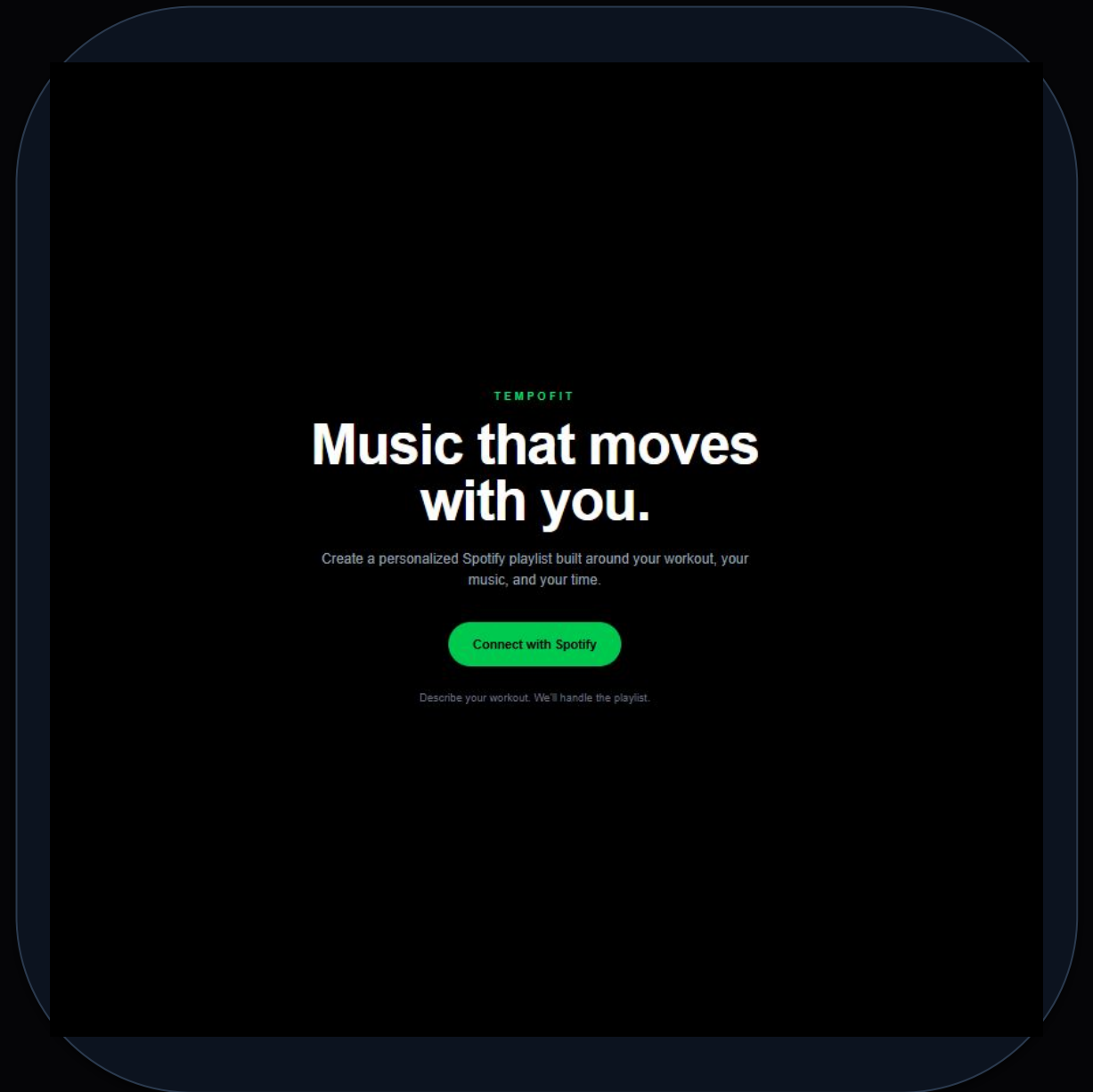
TEMPOFIT

Music that moves with you.

A Spotify-integrated workout playlist generator that turns natural-language workouts, music preferences, and target duration into a structured playlist.

Product case study • Next.js • TypeScript • Spotify Web API • OAuth

Role: Product strategy, UX, architecture, implementation, QA



Problem → Product Opportunity

TempoFit was scoped as a portfolio MVP: small enough to ship, but complete enough to demonstrate product and integration decisions.

THE PROBLEM

Workout playlists are often static. Users still have to search, sequence songs, and manually estimate whether a playlist fits the workout.

THE USER NEED

Describe the workout once, keep control over genre and familiarity, and get a playlist that roughly matches the workout's duration and phases.

THE MVP

Spotify connection + preferences + natural-language workout input + phase-based playlist generation + regenerate + save to Spotify.

Deliberate non-goals

- No Apple Music or multi-provider support
- No native mobile app, social features, wearables, or monetization
- No production database or ML recommendation model
- Focus: prove the end-to-end product experience and technical architecture

End-to-End Product Flow

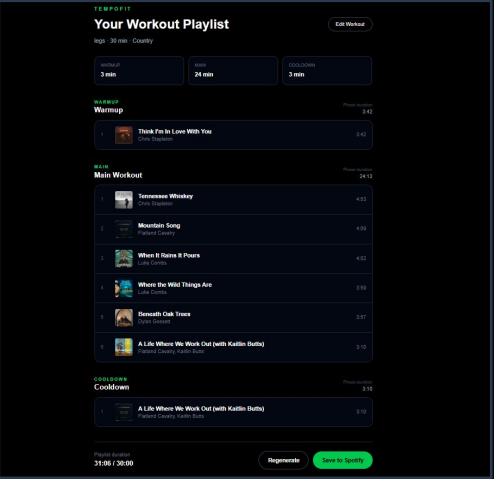
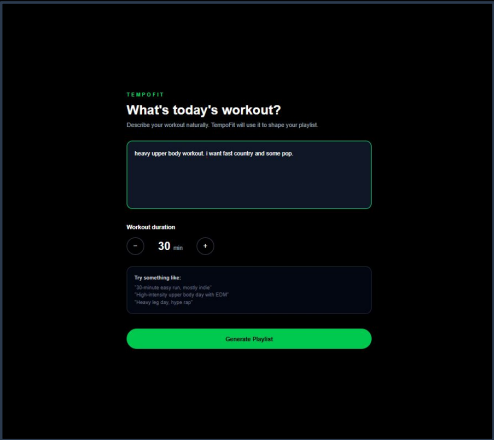
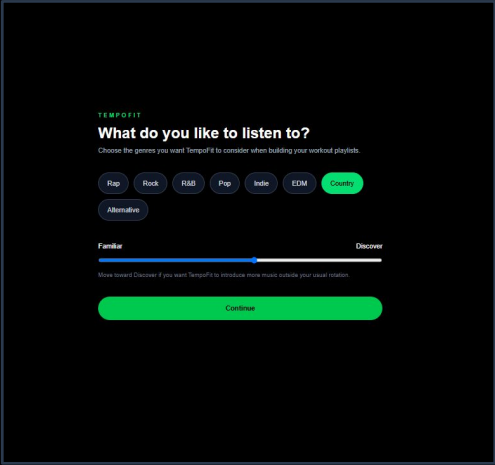
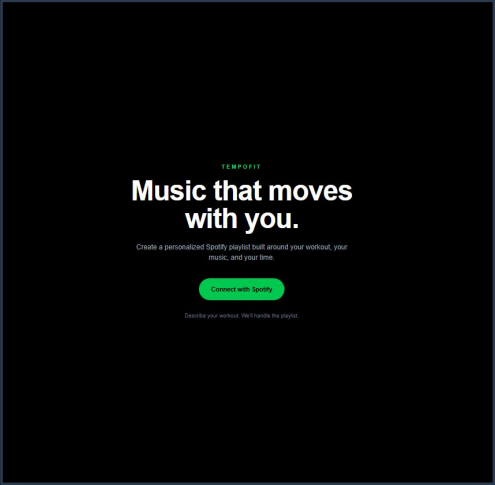
The experience intentionally stays linear: connect → personalize → describe workout → review → save.

1 CONNECT

2 PERSONALIZE

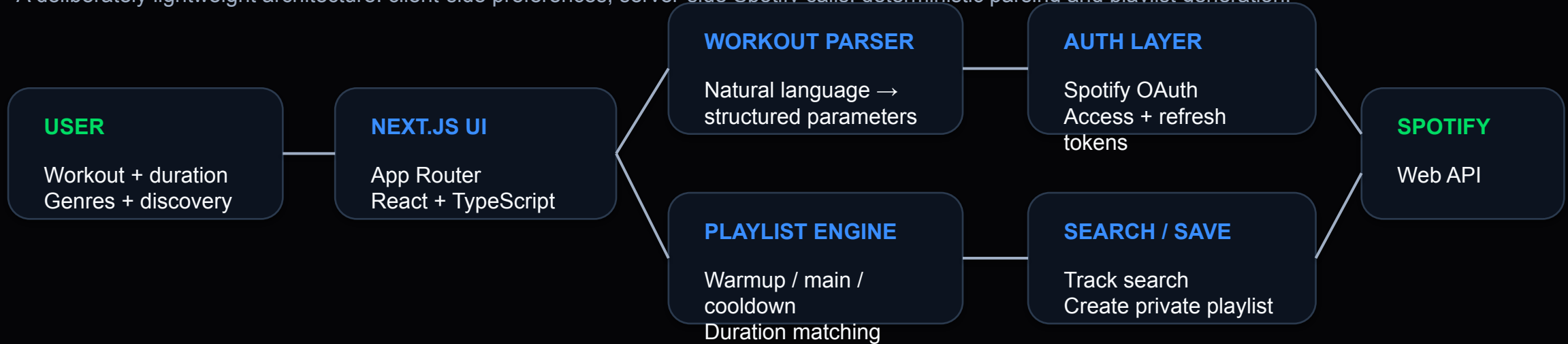
3 DESCRIBE

4 REVIEW + SAVE



System Architecture

A deliberately lightweight architecture: client-side preferences, server-side Spotify calls, deterministic parsing and playlist generation.



Key implementation choices

- Preferences stored in localStorage for the MVP; no database required.
- Spotify Client Secret remains server-side; OAuth uses a local redirect callback.
- Automatic refresh-token flow recovers when the access token expires.
- Recommendation behavior is rule-based and explainable rather than ML-driven.

Constraints → Engineering Decisions

The hardest part was not building more features; it was adapting the product when the external API could not support the original recommendation approach

	CONSTRAINT	DECISION	WHY IT MATTERED
01	Spotify recommendation / audio-analysis limitations	Reworked recommendation architecture around curated Familiar vs. Discover artist pools and Spotify Search.	Preserved user control without depending on restricted endpoints.
02	Workout duration is not equal to exact song lengths	Split playlists into warmup, main, and cooldown targets, then selected tracks with deterministic duration heuristics.	Generated playlists that stay close to the requested workout length.
03	OAuth access tokens expire	Implemented a refresh-token endpoint and retry path rather than forcing the user to reconnect.	Maintained the playlist flow after token expiration.
04	External API calls can fail or return weak results	Added fallback behavior, regeneration, and test coverage around search, auth, and playlist creation.	Made failures recoverable instead of dead-ending the experience.

Validation & QA

Testing focused on the flows most likely to break: duration matching, regeneration, authentication, and saving back to Spotify.

15–45 MIN

Playlist generation validated across short, medium, and longer workout targets.

TOKEN REFRESH

Manually removed the access cookie and verified automatic recovery.

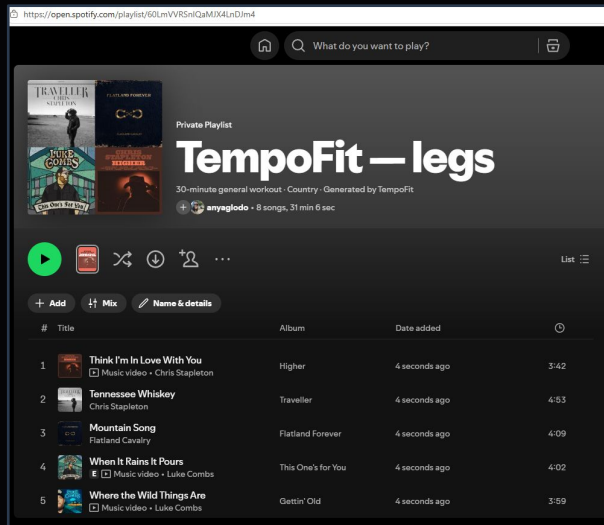
REGENERATION

Verified a user can request a new playlist without restarting the flow.

SPOTIFY SAVE

Created a private Spotify playlist and confirmed tracks were added successfully.

Example end-to-end result



30-minute Country workout

TempoFit generated and saved an 8-song Spotify playlist at 31:06 for a 30:00 target.

- Verified Familiar / Discover preference changes the candidate pool.
- Production build completed successfully.
- Secret scan confirmed no Spotify client secret was committed.
- Known limitation: heuristic matching aims for approximate—not exact—duration.

What I'd Build Next

The MVP proves the workflow. The next phase would focus on production readiness and richer personalization—not simply adding more screens.

01 PRODUCTION AUTH

Move beyond local portfolio OAuth/session handling with hardened session storage, deployment-safe redirects, and production monitoring.

02 PERSISTENT USER DATA

Add a database for preferences, playlist history, saved workout templates, and cross-device continuity.

03 BETTER PERSONALIZATION

Use permitted listening signals or richer user feedback to evolve beyond curated pools while keeping recommendations explainable.

04 OBSERVABILITY

Instrument API failures, auth errors, generation latency, and save success so integration health can be measured.

05 USER VALIDATION

Run structured usability tests and measure whether generated playlists actually reduce setup time and improve workout fit.

06 PLATFORM EXPANSION

Evaluate additional music providers only after the core Spotify experience and retention signals justify the added complexity.

Takeaway

TempoFit demonstrates end-to-end ownership: product scoping, UX, API integration, architecture tradeoffs, debugging, QA, and a working Spotify output.